EXERCISE 03: MATPLOTLIB DATA VISUALIZATION — COMPREHENSIVE STUDY GUIDE

College Data Science Laboratory Course — Practical Examination Study Guide
Source Code File: `e03_matplotlib.py` | **Repository:** `https://github.com/akhilkvu6/ds` | **Domain:** 2D Data Visualization, Statistical Graphics & Plotting

1. Program Information and Aim

- **Program Identity:** Program 03 — `e03_matplotlib.py` (Contains 10 Comprehensive Plotting Exercises).
- **Academic Aim:** To master 2D data visualization using Python's `matplotlib.pyplot` library. Covers single line charts with custom markers and styles, labeled business graphs, file-driven plotting from raw text files (`exercise3.3.txt`), multi-line comparative plots with legends, side-by-side subplots with super-titles, vertical, horizontal and multi-colored bar charts, grouped dual-category bar charts using mathematical x-offsets, proportional pie charts with percentage formatting, CSV-driven categorical pie charts (`medal.csv`), and multi-series comparative scatter plots.
- **Required Libraries & Supporting Files:**
  - `matplotlib.pyplot as plt`: Main visualization interface.
  - `numpy as np`: Coordinate array definitions and arithmetic sequence manipulations.
  - `pandas as pd`: Tabular data reading for Olympic medal analysis.
  - `exercise3.3.txt`: Local text file containing comma-separated Day and Temperature values.
  - `medal.csv`: Local CSV file containing Olympic country names and gold medal counts.

2. Prerequisites and Foundational Concepts

- **Matplotlib Anatomy:** A *Figure* is the outer canvas window. An *Axes* is the actual plotting region with coordinate scales, ticks, and labels. Elements plotted on an Axes include lines, markers, bars, wedges, and text annotations.
- **State-based vs Object-Oriented Interface:** `e03_matplotlib.py` utilizes the state-based `pyplot` interface (mimicking MATLAB), tracking the "current" figure and axes automatically.
- **Blocking Behavior of `plt.show()`:** When `plt.show()` is called in standard script execution, it opens a GUI window and pauses code execution until the user closes the window. Each of the 10 questions ends with `plt.show()`, creating 10 sequential visual windows.

3. Complete Exam-Ready Algorithms (START ... STOP Format)

MAIN OVERALL ALGORITHM: Exercise 03 (All 10 Plotting Questions)

Step 1: START
Step 2: [Import Libraries] Import `matplotlib.pyplot as plt`, `numpy as np`, and `pandas as pd`.
Step 3: [Q1: Styled Line Plot] Define coordinates $(1, 3)$, $(2, 10)$, $(6, 12)$, $(18, 20)$. Call `plt.plot(x, y, color="red", linestyle=":", marker=".", markersize=10, markerfacecolor="green", markeredgecolor="green")`. Display via `plt.show()`.
Step 4: [Q2: Temperature vs Sales] Define temperature array x and sales array y. Set axis labels using `plt.xlabel()` and `plt.ylabel()`. Plot with green color and `*` marker. Display via `plt.show()`.
Step 5: [Q3: File-Driven Line Plot] Open `exercise3.3.txt`. Parse line 1 for x and line 2 for y using `strip().split(",")` and `map(int, ...)`. Close file. Set title and labels. Plot with `*` marker. Display via `plt.show()`.
Step 6: [Q4: Multi-Line Plot with Legend] Define shared days array x, high temps $y1$, and low temps $y2$. Plot $y1$ with red line and label "Highest Temperature". Plot $y2$ with blue line and label "Lowest Temperature". Attach legend via `plt.legend()` and display.
Step 7: [Q5: Subplots] Create figure of size $(10, 4)$. Activate Subplot 1 via `plt.subplot(1, 2, 1)` and plot Highs. Activate Subplot 2 via `plt.subplot(1, 2, 2)` and plot Lows. Add overall title via `plt.suptitle()` and display.
Step 8: [Q6: Bar Charts (3 Variants)]
i. Vertical Bar Chart: `plt.bar(languages, popularity)`.
ii. Horizontal Bar Chart: `plt.barh(languages, popularity, color="red")`.
iii. Multi-Colored Bar Chart: `plt.bar(languages, popularity, color=colors)`.
Step 9: [Q7: Grouped Dual Bar Chart] Generate group centers $x = [3, 6, 9, 12, 15]$. Set `width = 0.4`. Plot Men bars shifted left: `plt.bar(x - width, men, width=width, label="Men")`. Plot Women bars shifted right: `plt.bar(x + width, women, width=width, label="Women")`. Attach legend and display.
Step 10: [Q8: Pie Chart from Lists] Plot language shares using `plt.pie(popularity, labels=languages, autopct="%1.1f%%")`. Add title and display.
Step 11: [Q9: CSV-Driven Olympic Pie Chart] Read `medal.csv` using `pd.read_csv()`. Extract Country and Gold_Medal columns as NumPy arrays. Plot via `plt.pie(medals, labels=countries, autopct="%1.1f%%")`. Add title and display.
Step 12: [Q10: Math vs Science Scatter Plot] Define student IDs 1 to 10. Plot Math marks in red and Science marks in green using `plt.scatter()`. Attach labels, title, legend, and display.
Step 13: STOP

4. Complete Theory, Mathematical Formulations, and Worked Examples

4.1 Grouped Bar Chart Geometric Offset Formula

To display two categories (Men and Women) side-by-side without overlapping:
• Group centers: $x = [3, 6, 9, 12, 15]$ (spacing of 3 units between groups).
• Bar width: $w = 0.4$.
• Men bar positions: $x_{\text{men}} = x - w = [2.6, 5.6, 8.6, 11.6, 14.6]$.
• Women bar positions: $x_{\text{women}} = x + w = [3.4, 6.4, 9.4, 12.4, 15.4]$.
Because the total occupied width is $2 \times 0.4 = 0.8$ units, and groups are separated by 3 units, bars remain neatly paired with clear visual separation.

4.2 Pie Chart Percentage Allocation Formula

For raw category values $v_1, v_2, \dots, v_k$: $$\text{Percentage } P_i = \left(\frac{v_i}{\sum_{j=1}^k v_j} \right) \times 100\%$$ • In Question 8: $\sum v_j = 30 + 40 + 10 + 10 + 5 + 5 = 100$. Python represents $40 / 100 = 40.0\%$.
• In Question 9 (Olympic Gold Medals from `medal.csv`):
USA: 46, UK: 27, China: 26, Russia: 19, Germany: 17 → Total medals = $46 + 27 + 26 + 19 + 17 = 135$.
USA Percentage: $\frac{46}{135} \times 100\% \approx 34.1\%$. UK Percentage: $\frac{27}{135} \times 100\% \approx 20.0\%$.
Format string `autopct="%1.1f%%"` displays floating-point percentages formatted to one decimal place.

5. Complete Original Source Code and Block Walkthrough

5.1 Verbatim Source Code (`e03_matplotlib.py`)

```
# Exercise 3: Programs using Matplotlib

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd


# Question 1: Draw a line from position (1,3) and (2,10)
# then to (6,12) and finally to (18,20).
# Mark each point with green color and set line colour to red
# and line style to dotted.

x = np.array([1, 2, 6, 18])
y = np.array([3, 10, 12, 20])

plt.plot(
    x, y,
    color="red",
    linestyle=":",
    marker=".",
    markersize=10,
    markerfacecolor="green",
    markeredgecolor="green"
)

plt.show()


# Question 2: Draw a plot for the following data:
# Temperature in degree and Sales.

x = np.array([12, 14, 16, 18, 20, 22, 24])
y = np.array([100, 200, 250, 400, 300, 450, 500])

plt.xlabel("Temperature")
plt.ylabel("Sales")

plt.plot(x, y, color="green", marker="*")

plt.show()


# Question 3: Draw a line using given axis values
# taken from a text file.

file = open("exercise3.3.txt")

x = np.array(list(map(int, file.readline().strip().split(","))))
y = np.array(list(map(int, file.readline().strip().split(","))))

file.close()

plt.title("Temperature Information")
plt.xlabel("Day")
plt.ylabel("Temperature")

plt.plot(x, y, marker="*")

plt.show()
```

```
# Question 4: Plot two or more lines on same plot
# with suitable legends.

x = np.array([1, 2, 3, 4, 5])

y1 = np.array([30, 32, 31, 32, 33])
y2 = np.array([27, 26, 29, 28, 29])

plt.plot(
    x, y1,
    color="red",
    marker=".",
    label="Highest Temperature"
)

plt.plot(
    x, y2,
    color="blue",
    marker=".",
    label="Lowest Temperature"
)

plt.xlabel("Day")
plt.title("Temperature Difference")
plt.legend()

plt.show()


# Question 5: Create multiple plots.

x = np.array([1, 2, 3, 4, 5])

y1 = np.array([30, 32, 31, 32, 33])
y2 = np.array([27, 26, 29, 28, 29])

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)

plt.plot(x, y1, color="red", marker=".")

plt.xlabel("Day")
plt.ylabel("Temperature (°C)")
plt.title("Highs")


plt.subplot(1, 2, 2)

plt.plot(x, y2, color="blue", marker=".")

plt.xlabel("Day")
plt.ylabel("Temperature (°C)")
plt.title("Lows")

plt.suptitle("Temperature Difference Over 5 Days")

plt.show()


# Question 6: Programming language and popularity.
# i. Bar chart
# ii. Horizontal bar chart
```

```
# iii. Bar chart with different colors.

languages = ["Java", "Python", "PHP", "JavaScript", "C#", "C++"]
popularity = [22.2, 17.6, 8.8, 8, 7.7, 6.7]


# i. Bar chart

plt.bar(languages, popularity)

plt.xlabel("Programming Languages")
plt.ylabel("Popularity")

plt.show()


# ii. Horizontal bar chart

plt.barh(languages, popularity, color="red")

plt.xlabel("Popularity")
plt.ylabel("Programming Languages")
plt.title("Programming Languages Popularity")

plt.show()


# iii. Different colors for each bar

colors = [
    "red",
    "green",
    "blue",
    "orange",
    "purple",
    "yellow"
]

plt.bar(languages, popularity, color=colors)

plt.xlabel("Programming Languages")
plt.ylabel("Popularity")

plt.show()


# Question 7: Create bar plots of scores by group and gender.
# Use multiple x values on the same chart for men and women.

x = np.arange(1, 6) * 3

men = np.array([22, 30, 35, 35, 26])
women = np.array([25, 22, 30, 35, 29])

width = 0.4

plt.bar(
    x - width,
    men,
    width=width,
    label="Men"
)

plt.bar(
    x + width,
```

```
    women,
    width=width,
    label="Women"
)

plt.xlabel("Group")
plt.ylabel("Score")
plt.title("Scores by Group and Gender")
plt.legend()

plt.show()


# Question 8: Create a pie chart on the popularity
# of programming languages.

languages = [
    "Java",
    "Python",
    "PHP",
    "JavaScript",
    "C#",
    "C++"
]

popularity = [30, 40, 10, 10, 5, 5]

plt.pie(
    popularity,
    labels=languages,
    autopct="%1.1f%%"
)

plt.title("Programming Languages Popularity")

plt.show()


# Question 9: Create a pie chart of gold medal achievements
# of five most successful countries in 2016 Summer Olympics.
# Read the data from a CSV file.

df = pd.read_csv("medal.csv")

countries = df["Country"].to_numpy()
medals = df["Gold_Medal"].to_numpy()

plt.pie(
    medals,
    labels=countries,
    autopct="%1.1f%%"
)

plt.title("Gold Medals - 2016 Summer Olympics")

plt.show()


# Question 10: Draw a scatter plot comparing
# Mathematics and Science marks of 10 students.

math_marks = np.array([
    88, 92, 80, 89, 100,
    80, 60, 100, 80, 34
])
```

```
science_marks = np.array([
    35, 79, 79, 48, 100,
    88, 32, 45, 20, 30
])

students = np.arange(1, 11)

plt.scatter(
    students,
    math_marks,
    color="red",
    label="Mathematics"
)

plt.scatter(
    students,
    science_marks,
    color="green",
    label="Science"
)

plt.xlabel("Students")
plt.ylabel("Marks")
plt.title("Mathematics vs Science Marks")
plt.legend()

plt.show()
```

6. Complete Function, Parameter, and Syntax Reference

6.1 Comprehensive 10-Point Technical Breakdown

1. `plt.plot(x, y, ...)`: Renders 2D connected line graphs. Key styling parameters: `color="red"`, `linestyle=":"` (dotted), `marker="."` or `"*"`, `markersize=10`, `markerfacecolor`, `markeredgecolor`.
2. `plt.xlabel()`, `plt.ylabel()`, `plt.title()`: Text annotation functions adding titles and axis descriptions to the current active axes.
3. `file.readline().strip().split(",")` & `map(int, ...)`: String parsing idiom used in Question 3 to ingest comma-delimited numeric coordinates from plain text files.
4. `plt.legend()`: Automatically creates an explanatory box associating visual styles with text descriptions defined by `label="..."` in preceding plot calls.
5. `plt.figure(figsize=(width, height))`: Initializes a new canvas figure with dimensions in inches. Line 98 creates a 10×4 canvas for subplots.
6. `plt.subplot(nrows, ncols, index)`: Subdivides figure into an $m \times n$ grid. `plt.subplot(1, 2, 1)` activates panel 1; `plt.subplot(1, 2, 2)` activates panel 2.
7. `plt.suptitle(t)`: Adds a centralized super-title spanning across all subplots on the entire figure canvas.
8. `plt.bar(x, height, width=0.8, color=None)`: Creates vertical bar charts. Accepts categorical strings or numeric coordinates. `color` can be a single color name or a list of colors per bar.
9. `plt.barh(y, width, color=None)`: Creates horizontal bar charts where bars extend horizontally along the x-axis.
10. `plt.pie(x, labels=None, autopct=None)`: Generates proportional circular pie charts. `autopct="%1.1f%%"` formats slice percentages automatically.
11. `df["Column"].to_numpy()`: Converts Pandas Series columns into pure NumPy 1D ndarrays for seamless passing to Matplotlib functions.
12. `plt.scatter(x, y, color=None, label=None)`: Plots individual un-connected coordinate points in 2D Cartesian space.

6.2 Technical Coverage Checklist

Technical Concept / Item	Line Number	Status	Exam Relevance
Custom Line Styling & Markers	Lines 16–24	✓ Verified	Dotted lines, custom face/edge marker colors
Axis Labels and Plot Titles	Lines 35–36, 53–55	✓ Verified	`xlabel`, `ylabel`, `title`
Text File Ingestion via `strip().split()`	Lines 48–49	✓ Verified	External file coordinate parsing
Multi-Line Overlay with Legend	Lines 70–86	✓ Verified	`label` parameter and `plt.legend()`
Side-by-Side Subplots & Suptitle	Lines 98–117	✓ Verified	`plt.subplot(1, 2, 1)` & `plt.suptitle()`
Vertical, Horizontal & Colored Bars	Lines 133, 143, 163	✓ Verified	`bar`, `barh`, list of color strings
Grouped Dual-Category Bars (Offsets)	Lines 181–193	✓ Verified	`x - width` and `x + width` logic
Categorical Pie Charts with `autopct`	Lines 217–221	✓ Verified	Percentage string format `"%1.1f%%"`
CSV Data Extraction to NumPy	Lines 232–236	✓ Verified	`pd.read_csv()` and `.to_numpy()`
Multi-Series Scatter Plotting	Lines 263–275	✓ Verified	`plt.scatter()` with distinct colors and labels

7. Step-by-Step Program Execution Walkthrough

Exercise	Type of Visual Plot	Input Data & Sources	Visual Characteristics
Q1	Styled Line Plot	$x=[1, 2, 6, 18]$, $y=[3, 10, 12, 20]$	Red dotted line with green dots
Q2	Business Line Plot	Temperature vs Sales arrays	Green line with star markers, labeled axes
Q3	File Line Plot	`exercise3.3.txt` (Day & Temp)	Line with star markers from parsed file values
Q4	Comparative Multi-Line	Day vs High/Low temperatures	Red line (Highs) and Blue line (Lows) with Legend
Q5	Multi-Panel Subplots	Two $(1, 2)$ grid subplots	Left panel: Highs; Right panel: Lows; Super-title above
Q6	Bar Chart Trio	6 Programming Languages	(i) Default blue bars, (ii) Red horizontal bars, (iii) 6 distinct bar colors
Q7	Grouped Dual Bars	5 groups × Men/Women	Paired bars at $x \pm 0.4$ with Legend
Q8	Proportional Pie Chart	6 Language popularity shares	Circular wedges with percentages (e.g. 40.0%, 30.0%)
Q9	CSV-Driven Pie Chart	`medal.csv` (5 Countries)	Olympic gold medal proportions (USA 34.1%, UK 20.0%, etc.)
Q10	Dual-Series Scatter Plot	10 Students × Math & Science	Red dots (Math) and Green dots (Science)

8. Output and Practical Interpretation

8.1 Visual Figure Windows Description

Summary of the 10 Displayed Graphic Windows:
• **Window 1:** Sharp zigzag red dotted line connecting coordinates $(1,3)$ to $(18,20)$, punctuated by solid green circle markers.
• **Window 2:** Upward-trending sales curve marked with green asterisks.
• **Window 3:** Day vs Temperature line plot extracted from `exercise3.3.txt`.
• **Window 4:** Two parallel lines on a single plot: red for highs and blue for lows, accompanied by an explanatory legend.
• **Window 5:** Clean side-by-side subplot canvas $(10 \times 4\text{ in})$ displaying Highs on the left and Lows on the right with a bold super-title.
• **Window 6:** Three successive bar charts displaying programming language popularities vertically, horizontally, and multi-colored.
• **Window 7:** Clustered bar chart showing side-by-side comparison of Men vs Women across 5 experimental groups.
• **Window 8:** Language popularity pie chart where Python occupies the largest wedge (40.0%), followed by Java (30.0%).
• **Window 9:** Olympic gold medal pie chart showing USA leading with 34.1% of medals, followed by UK (20.0%) and China (19.3%).
• **Window 10:** Scatter plot comparing student performance in Mathematics (red) vs Science (green) across 10 students.

9. Exam Preparation: Lab Record Answer and Model Questions

9.1 Lab Record Standard Format (Write in 1 Page)

EXERCISE 3: PROGRAMS USING MATPLOTLIB

AIM: To write a Python program using Matplotlib to create diverse 2D visual charts including styled line plots, file-driven charts, multi-line graphs with legends, subplots, bar charts (vertical, horizontal, multi-colored, grouped), pie charts from lists and CSV files, and multi-series scatter plots.

ALGORITHM:
1. Import `matplotlib.pyplot as plt`, `numpy as np`, and `pandas as pd`.
2. Plot styled line graphs using `plt.plot()` with `color`, `linestyle`, and `marker` options.
3. Parse coordinate data from text files using Python file I/O methods and plot.
4. Overlay multiple series on single axes using distinct labels and call `plt.legend()`.
5. Create side-by-side figure subplots using `plt.subplot(1, 2, 1)` and `plt.suptitle()`.
6. Render vertical and horizontal bar charts using `plt.bar()` and `plt.barh()`.
7. Implement grouped bars by applying mathematical offsets: $x - w$ and $x + w$.
8. Construct pie charts using `plt.pie()` with `autopct="%1.1f%%"`.
9. Read CSV data via `pd.read_csv()`, convert columns to NumPy arrays, and plot.
10. Render dual-category scatter plots using `plt.scatter()` and display via `plt.show()`.

RESULT: All 10 graphical visualization programs were successfully implemented, executed, and verified.

9.2 Common Exam Questions and Model Answers

- **2-Mark Question:** What is the purpose of `autopct="%1.1f%%"` in `plt.pie()`?
  Answer: It formats and displays the percentage label for each pie wedge as a floating-point number rounded to one decimal place, followed by a literal percent sign.
- **3-Mark Question:** Explain how grouped bar charts are constructed without overlapping.
  Answer: Grouped bar charts apply symmetric mathematical coordinate offsets to the category centers (x). By plotting the first series at $x - \text{width}$ and the second series at $x + \text{width}$, both bars stand side-by-side without overlapping.
- **5-Mark Question:** Differentiate between `plt.subplot()` and `plt.suptitle()`, and explain the three integer arguments in `plt.subplot(1, 2, 1)`.

Answer: `plt.subplot(nrows, ncols, index)` divides the figure window into a grid of $nrows \times ncols$ panels and activates the panel at position `index` (1-indexed). `plt.subplot(1, 2, 1)` creates a 1-row by 2-column grid and activates the 1st (left) panel. In contrast, `plt.suptitle()` attaches an overarching, centralized title across the entire figure canvas spanning all subplots.

9.3 Predict-the-Output Questions

1. **Question:** What visual element does `linestyle=":"` produce in `plt.plot()`?
   Answer: A dotted line.
2. **Question:** If `plt.subplot(2, 3, 4)` is executed, how many total subplots are configured in the grid?
   Answer: 6 total subplots ($2 \text{ rows} \times 3 \text{ columns}$), activating the 4th panel (first panel of the second row).

9.4 Identify-the-Function Questions

1. **Question:** Which function in `matplotlib.pyplot` creates a horizontal bar chart?
   Answer: `plt.barh()`.
2. **Question:** Which function is used to display an explanatory legend for labeled lines or bars?
   Answer: `plt.legend()`.
3. **Question:** Which function renders un-connected individual data points in 2D Cartesian space?
   Answer: `plt.scatter()`.

10. Comprehensive Viva Voce Questions and Answers (16 Exam Questions)

Q1: What is the difference between `plt.bar()` and `plt.barh()`?
Answer: `plt.bar()` draws vertical bars where categories are on the x-axis and heights are on the y-axis. `plt.barh()` draws horizontal bars where categories are on the y-axis and lengths extend along the x-axis.

Q2: Why must we call `plt.legend()` after specifying `label="..."` in plot calls?
Answer: The `label` parameter merely registers a label string with the plotted artist object. `plt.legend()` must be explicitly called to render the visible legend box on the axes.

Q3: What does the `figsize` parameter in `plt.figure(figsize=(10, 4))` represent?
Answer: It defines the figure canvas dimensions in inches as a tuple: $(\text{width}, \text{height}) = (10\text{ inches}, 4\text{ inches})$.

Q4: How does `plt.show()` behave during script execution?
Answer: It launches the graphical GUI window and blocks further script execution until the user manually closes the figure window.

Q5: What is the function of `plt.tight_layout()`?
Answer: It automatically adjusts subplot parameters and margins to prevent axis labels, titles, and ticks from overlapping neighboring subplots.

Q6: How do you pass a unique color to each bar in a bar chart?
Answer: By passing a Python list of color strings (e.g. `color=["red", "green", "blue", ...]`) matching the length of the categories into `plt.bar()`.

Q7: What does the format specifier `%1.1f%%` mean in `autopct`?
Answer: `%1.1f` formats a floating-point number with one decimal place. The double percent `%%` escapes to print a literal `%` character.

Q8: What is the difference between a line plot (`plt.plot`) and a scatter plot (`plt.scatter`)?
Answer: `plt.plot()` connects sequential data points with continuous line segments, whereas `plt.scatter()` plots isolated individual markers without connecting lines.

Q9: How do you set different colors for the interior and boundary of a line marker?
Answer: Using `markerfacecolor` for the interior fill color and `markeredgecolor` for the outer border outline.

Q10: Why is `df["Country"].to_numpy()` used before plotting with Matplotlib?
Answer: While Matplotlib supports Pandas Series, converting to pure NumPy ndarrays ensures maximum compatibility, eliminates indexing metadata, and prevents dimensional type warnings.

Q11: What is the purpose of `file.close()` after reading coordinates from a text file?
Answer: It releases system file descriptor handles and flushes any pending I/O buffers, preventing memory leaks and locked file locks.

Q12: In grouped bar charts, why do we multiply `np.arange(1, 6) * 3`?
Answer: To establish category centers at $3, 6, 9, 12, 15$, providing ample horizontal spacing (3 units) between distinct group clusters so bars do not crowd each other.

Q13: What is the default marker style in `plt.plot()` if no marker argument is provided?
Answer: None (a solid continuous line is drawn without point markers).

Q14: Can a pie chart display slices that do not sum to 100?
Answer: Yes. Matplotlib automatically normalizes raw values: $\frac{x_i}{\sum x}$ so that wedges always form a complete 360° circle.

Q15: What is the difference between `plt.title()` and `plt.suptitle()`?
Answer: `plt.title()` sets the title for an individual active subplot/axes. `plt.suptitle()` sets an overarching master title for the entire figure window above all subplots.

Q16: How is Matplotlib utilized in subsequent Machine Learning lab exercises?
Answer: In KNN (Exercise 05) to visualize decision classifications, in Decision Trees (Exercise 07) via `tree.plot_tree()`, in K-Means (Exercise 09) to plot clusters and centroids, and in Multiple Linear Regression (Exercise 10) to plot fit lines and diagnostic residuals.

11. Common Mistakes, Pitfalls, and Debugging Guide

Common Error / Pitfall	Root Cause	Correction / Debugging Technique
Legend not appearing on chart	Calling `plt.legend()` without defining `label="..."` in preceding plot calls.	Add `label="Series Name"` to each plot/bar statement before calling `plt.legend()`.
Subplots overlapping labels and titles	Omitting spacing management in multi-panel figures.	Call `plt.tight_layout()` before `plt.show()` to auto-adjust margins.
Program freezes at `plt.show()`	Assuming background execution while GUI window is open.	`plt.show()` is blocking by default. Close the figure window to proceed to the next exercise.
Grouped bars overlapping each other	Plotting both categories at the exact same x coordinates.	Offset x-coordinates: `x - width` for the first group and `x + width` for the second group.
Wrong delimiter when parsing `exercise3.3.txt`	Splitting by whitespace instead of commas.	The file contains comma-separated values: use `.strip().split(",")`.

12. Memory Tricks, Mnemonics, and Last-Minute Rapid Revision

12.1 Mnemonics & Memory Aids

- **Mnemonic for Line Styling Parameters — "C-L-M-S-F-E":**
  Color • **L**inestyle • **M**arker • **M**arker**S**ize • **M**arker**F**acecolor • **M**arker**E**dgecolor.
- **Mnemonic for Grouped Bar Offsets:** "Minus for Men, Plus for Women" — `x - width` and `x + width`.
- **Pie Chart Formatting:** `autopct="%1.1f%%"` — Float with 1 decimal + literal percent.

12.2 Top 5 High-Priority Viva Questions (Last-Minute Refresh)

1. What does `plt.subplot(1, 2, 2)` mean?
Answer: A grid of 1 row and 2 columns, activating the 2nd (right) subplot panel.

2. Why must we call `plt.legend()`?
Answer: To make labels registered via `label="..."` visibly appear in an explanatory legend box.

3. How are grouped bars prevented from overlapping?
Answer: By subtracting and adding the bar width from group centers: $x - \text{width}$ and $x + \text{width}$.

4. What is the difference between `plt.bar()` and `plt.barh()`?
Answer: `plt.bar()` creates vertical bars; `plt.barh()` creates horizontal bars.

5. What is the difference between `plt.title()` and `plt.suptitle()`?
Answer: `plt.title()` titles a single subplot; `plt.suptitle()` sets a figure-wide title across all subplots.

13. Cross-Program Understanding (Data Science Pipeline Progression)

Matplotlib serves as the visual communication engine of the entire Data Science curriculum:

Exercise	Dataset / Method	Matplotlib Visual Implementation
Exercise 01 (NumPy)	Array generation	Visual foundation for array coordinates
Exercise 04 (Pandas)	DataFrames	Visualizing DataFrame columns via Pandas `.plot()` wrapper
Exercise 03 (Matplotlib)	**Synthetic & File Data**	**Lines, Bars, Subplots, Pie Charts, and Scatter Plots**
Exercise 05 (KNN)	Iris dataset	Two subplots comparing full dataset vs classification errors
Exercise 07 (Decision Tree)	Iris dataset	`tree.plot_tree()` for full flowchart visual graph
Exercise 09 (K-Means)	Synthetic Blobs	Cluster color scatter (`viridis`) and red 'x' centroid marks
Exercise 10 (Linear Regression)	Diabetes dataset	Side-by-side feature subplots & Actual vs Predicted diagonal line